

A Course End Project Report on

Bank Account Management System

Object-Oriented Programming through Java

Submitted by

Batch No. 7

Student Name: B Nithin

Roll Number: 26885A0547

Course Facilitator

Mr. Ganesh Deshmukh
OOP (A9601), OOPL (A9602)

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

Abstract . . . 2
1 Introduction . . . 2
2 Problem Statement . . . 2
3 Objectives . . . 2
4 System Requirements . . . 2
4.1 Hardware . . . 2
4.2 Software . . . 2
5 System Design . . . 3
5.1 Class Responsibilities . . . 3
5.2 Program Flow . . . 3
6 Implementation . . . 3
6.1 Account Creation . . . 3
6.2 Balance, Deposit, and Withdrawal . . . 3
6.3 Money Transfer . . . 3
6.4 Transaction History . . . 3
7 Object-Oriented Concepts Used . . . 4
8 Testing and Results . . . 4
9 Limitations and Future Enhancements . . . 4
10 Conclusion . . . 5

References **6**

Abstract

The Bank Account Management System is a console-based Java application designed to demonstrate object-oriented programming through common banking tasks. The user can create accounts, check balances, deposit and withdraw funds, transfer money, and view transaction history. The program separates its responsibilities among the `Main`, `Bank`, `BankAccount`, and transaction classes. Encapsulation protects account data, while inheritance and runtime polymorphism are demonstrated using an abstract transaction class and concrete deposit and withdrawal transactions. Input validation and exception handling reject invalid operations. This project is an educational prototype: account data is stored in memory and is lost when the program exits.

Keywords: Java, OOP, encapsulation, abstraction, inheritance, polymorphism, banking.

1 Introduction

Object-oriented programming organizes software around classes and objects. A banking system is a suitable learning example because it contains entities with data (accounts) and operations (deposits, withdrawals, and transfers). This project provides a menu-based Java application that models these operations and illustrates how OOP concepts improve separation of responsibilities.

2 Problem Statement

Manual tracking of account balances and transactions can lead to errors. A simple computerized model can organize account information, validate requests, and record activity during one program session. The goal is to implement a manageable educational system, not a real banking service.

3 Objectives

- Create bank accounts with automatically assigned account numbers.
- Provide balance enquiry, deposits, withdrawals, and money transfers.
- Maintain transaction history for each account during the current session.
- Demonstrate encapsulation, abstraction, composition, inheritance, and polymorphism.
- Handle invalid amounts, incorrect PINs, missing accounts, and insufficient balances.

4 System Requirements

4.1 Hardware

A computer capable of running a Java Development Kit and a terminal or Java IDE.

4.2 Software

Java Development Kit (JDK) 17 or later; terminal/command prompt or an IDE such as IntelliJ IDEA, Eclipse, or Visual Studio Code with Java support.

5 System Design

5.1 Class Responsibilities

- **Main:** Displays the console menu, reads user input, authenticates account operations, and reports results.
- **Bank:** Creates and looks up accounts, manages the account collection, and coordinates transfers.
- **BankAccount:** Encapsulates account number, holder name, PIN, balance, and transaction history. It validates and performs deposits and withdrawals.
- **BankTransaction:** Abstract parent class that validates a positive transaction amount and defines `execute()`.
- **DepositTransaction and WithdrawalTransaction:** Concrete transaction classes that override `execute()` to perform the corresponding operation.

5.2 Program Flow

The application displays a menu and waits for the user's choice. For protected operations, it requests an account number and PIN. The system validates input, performs the requested operation, displays the result, and returns to the menu. Selecting Exit ends the program.

6 Implementation

6.1 Account Creation

The user enters a holder name, a four-digit PIN, and an opening balance. The `Bank` class generates an account number and stores the new `BankAccount` object.

6.2 Balance, Deposit, and Withdrawal

The balance enquiry displays the current balance after PIN verification. Deposit and withdrawal requests must have a positive, finite amount. Withdrawal is rejected when the amount exceeds the available balance.

6.3 Money Transfer

The `Bank` class checks that both accounts exist and are different, then withdraws from the source and deposits into the destination. The program rejects non-positive amounts and insufficient funds.

6.4 Transaction History

Each account records its opening balance and transaction activity for the duration of the program. A user must authenticate before viewing the account's history.

7 Object-Oriented Concepts Used

Concept	Application in the project
Encapsulation	Account fields are private. Public methods validate requests and control access to balance-changing operations.
Abstraction	The menu and `Bank` methods expose useful operations without revealing internal storage details.
Composition	The `Bank` object manages a collection of `BankAccount` objects.
Inheritance	`DepositTransaction` and `WithdrawalTransaction` extend the abstract `BankTransaction` class.
Runtime polymorphism	`BankAccount.executeTransaction()` accepts the abstract parent type; the overridden `execute()` method is selected for the concrete transaction object.
Exception handling	Invalid amounts, incorrect PINs, missing accounts, and insufficient funds are handled with meaningful messages.

8 Testing and Results

The Java source and test harness compile with JDK 17 or later. The test harness uses Java assertions and reports the total number of checks that pass.

Test case	Action	Expected result
Account creation	Create an account with valid details	Account is created and assigned a number.
Deposit	Deposit a positive amount	Balance increases.
Invalid deposit	Deposit zero or a negative amount	Request is rejected and balance is unchanged.
Withdrawal	Withdraw an amount within available balance	Balance decreases.
Insufficient funds	Withdraw more than the balance	Request is rejected and balance is unchanged.
Transfer	Transfer between two valid accounts	Source is debited and destination is credited.
PIN verification	Compare correct and incorrect PIN values	Correct PIN passes; incorrect PIN fails.
Account lookup	Request an account that does not exist	An error is raised and reported.
Polymorphic transaction	Execute a deposit through the abstract transaction type	The deposit implementation runs and updates the balance.

The project was compiled and exercised using an assertion-based test harness. The repository's README documents the commands required to compile the application and run the tests.

9 Limitations and Future Enhancements

Accounts are stored only in memory; therefore, the information is lost when the program exits. PINs are held as plain strings and account listing is a demonstration feature rather than an authenticated administrator function. This software is not suitable for real financial use. Future work could add database persistence, hashed PINs, role-based authorization, stronger transfer atomicity, logging, and a graphical user interface.

10 Conclusion

The Bank Account Management System demonstrates how classes and object-oriented concepts can be applied to a familiar problem. It implements basic account operations with validation and transaction history while providing examples of encapsulation, abstraction, composition, inheritance, and runtime polymorphism.

References

[1] Oracle, *Java Documentation*. Available at: `https://docs.oracle.com/en/java/`

[2] Oracle, *The Java Tutorials: Object-Oriented Programming Concepts*. Available at: `https://docs.oracle.com/javase/tutorial/java/concepts/`

[3] Nithin-in, *Bank Account Management System, Batch 7*. Source code repository: `https://github.com/Nithin-in/Bank-Account-Management-System`